

Credit Card Fraud Detection using TensorFlow

Professional Project Report

Prepared By: Rramandeip Singh

Framework: TensorFlow & Keras

Language: Python

Dataset: Credit Card Fraud Detection (Kaggle)

Introduction

Credit card fraud has become one of the most significant challenges in the financial industry due to the rapid growth of online banking, e-commerce, and digital payment systems. Every day, financial institutions process millions of transactions, making it increasingly difficult to identify fraudulent activities using traditional rule-based methods. Fraudulent transactions not only result in substantial financial losses for banks and businesses but also reduce customer trust and increase operational costs. As the volume of transaction data continues to grow, artificial intelligence and deep learning have become essential tools for detecting suspicious patterns and improving fraud prevention systems.

This project focuses on developing a deep learning-based credit card fraud detection model using TensorFlow and Keras. The objective is to build a binary classification model capable of distinguishing between legitimate and fraudulent transactions by learning complex patterns from historical transaction data. Unlike conventional machine learning approaches that often require extensive manual feature engineering, deep neural networks automatically learn meaningful representations from the input data, making them highly effective for complex classification problems.

The project follows a structured end-to-end machine learning workflow. It begins with importing the required Python libraries and loading the publicly available Credit Card Fraud Detection dataset obtained from Kaggle. The dataset is then explored to understand its structure, feature distribution, and class imbalance. Data preprocessing techniques such as feature scaling are applied to improve model performance and ensure numerical stability during training.

After preprocessing, the dataset is divided into training and testing subsets. A Sequential Neural Network is then designed using TensorFlow and Keras, consisting of multiple fully connected (Dense) layers and Dropout layers to reduce the risk of overfitting. The model is compiled using the Adam optimizer and the Binary Cross-Entropy loss function, which are well suited for binary classification tasks such as fraud detection.

The neural network is trained over multiple epochs, during which both training and validation performance are monitored. After training, the model is evaluated using several performance metrics, including accuracy, confusion matrix, and classification report. In addition, training and validation accuracy and loss curves are generated to visualize the learning process and assess the model's ability to generalize to unseen data.

The trained model achieved approximately 99.93% testing accuracy, demonstrating its ability to correctly classify the majority of transactions. However, because the dataset is highly imbalanced, additional evaluation metrics such as precision, recall, and F1-score are also considered to provide a more comprehensive assessment of the model's fraud detection capability.

Overall, this project demonstrates the complete implementation of a deep learning solution for credit card fraud detection using Python and TensorFlow. It highlights essential concepts including data preprocessing, neural network design, model training, evaluation, visualization, and model persistence. The project also serves as a practical portfolio example showcasing skills in Python, TensorFlow, Keras, Scikit-learn, Pandas, NumPy, Matplotlib, and deep learning, making it suitable for academic submissions as well as GitHub and professional portfolios.

Project Objectives

- Develop a binary classification model using TensorFlow.
- Preprocess the dataset for effective model training.
- Evaluate the model using multiple performance metrics.
- Visualize the learning process with accuracy and loss curves.
- Save the trained model for future deployment.

Dataset Overview

Attribute	Details
Dataset	Credit Card Fraud Detection
Total Records	284,807
Features	30 input features
Target	Class (0 = Legitimate, 1 = Fraud)
Framework	TensorFlow / Keras
Problem Type	Binary Classification

Project Workflow

1. Import Libraries & Load Dataset
2. Data Preprocessing
3. Build Neural Network
4. Compile Model
5. Train Model
6. Evaluate Model
7. Generate Predictions
8. Visualize Results
9. Save Model

Implementation and Results

Import Libraries and Dataset Exploration

The dataset is loaded into a Pandas DataFrame and inspected using `head()`, `shape()`, `info()`, and `value_counts()`. This confirms the data structure, feature types, and highlights the class imbalance between legitimate and fraudulent transactions.

```
[13]: # Import Libraries
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

[14]: # Import Machine Learning Utilities
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

[53]: # Load Dataset
df = pd.read_csv(r"C:\Users\rnama_t5uau6\Downloads\archive (12)\creditcard.csv")
print(df.head())
print(df.shape)
print(df.info())
print(df['Class'].value_counts())
```

	Time	V1	V2	V3	V4	V5	V6	V7	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	

	V8	V9	...	V21	V22	V23	V24	V25	\
0	0.090698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	
4	-0.770533	0.817759	...	-0.009431	0.790278	-0.137458	0.141267	-0.206018	

Figure: Import Libraries and Dataset Exploration

Data Preprocessing

Data preprocessing is one of the most important stages of any machine learning or deep learning project because the quality of the input data directly affects the model's performance. In this project, the credit card transaction dataset was prepared to ensure that it could be effectively used for training the TensorFlow neural network.

The first step involved separating the dataset into input features (X) and the target variable (y). The input features include all transaction-related attributes such as Time, Amount, and the anonymized variables V1 to V28, while the target variable Class represents the transaction label. A value of 0 indicates a legitimate transaction, whereas 1 represents a fraudulent transaction.

After separating the features and labels, the Time and Amount columns were standardized using StandardScaler from the Scikit-learn library. These two features have values that differ significantly in scale compared to the remaining variables, which have already been transformed using Principal Component Analysis (PCA). Large differences in feature values can negatively influence the learning process because neural networks are sensitive to the scale of input data.

Standardization transforms the selected features so that they have a mean of zero and a standard deviation of one. This normalization ensures that all input features contribute more equally during model training. As a result, the optimization algorithm converges more efficiently, training becomes more stable, and the model is less likely to become biased toward features with larger numerical values.

Proper preprocessing also helps improve the overall performance of the deep learning model by reducing training time, minimizing numerical instability, and enabling the neural network to learn meaningful patterns more effectively. By preparing the dataset in this manner, the model is better equipped to distinguish between legitimate and fraudulent transactions, leading to more reliable predictions on previously unseen data.

Load the Dataset

The credit card transaction dataset is loaded into a Pandas DataFrame using the `read_csv()` function. After loading the data, the first few records, dataset dimensions, column information, and class distribution are displayed to understand the structure of the dataset.

```
[53]: # Load Dataset

df = pd.read_csv(r"C:\Users\rrama_t5uaug6\Downloads\archive (12)\creditcard.csv")

print(df.head())

print(df.shape)

print(df.info())

print(df['Class'].value_counts())
```

	Time	V1	V2	V3	V4	V5	V6	V7	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	

	V8	V9	...	V21	V22	V23	V24	V25	\
0	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	
4	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	

	V26	V27	V28	Amount	Class
0	-0.189115	0.133558	-0.021053	149.62	0
1	0.125895	-0.008983	0.014724	2.69	0
2	-0.139097	-0.055353	-0.059752	378.66	0
3	-0.221929	0.062723	0.061458	123.50	0
4	0.502292	0.219422	0.215153	69.99	0


```
[5 rows x 31 columns]
(284807, 31)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Time        284807 non-null float64
1   V1          284807 non-null float64
2   V2          284807 non-null float64
3   V3          284807 non-null float64
4   V4          284807 non-null float64
5   V5          284807 non-null float64
6   V6          284807 non-null float64
7   V7          284807 non-null float64
```

Figure: Data Preprocessing

Model Architecture

The core of this project is a deep neural network developed using the TensorFlow and Keras Sequential API. A Sequential model was selected because it is well suited for feedforward neural networks where data flows through the layers in a single direction, from the input layer to the output layer. This architecture is simple to implement, computationally efficient, and highly effective for binary classification problems such as credit card fraud detection.

The model begins with an Input layer, which accepts all preprocessed transaction features from the training dataset. Each transaction is represented by a set of numerical values, including the standardized Time and Amount features along with the anonymized variables V1 to V28. These features serve as the

input information from which the neural network learns patterns associated with legitimate and fraudulent transactions.

The first hidden layer consists of 64 neurons and uses the Rectified Linear Unit (ReLU) activation function. ReLU introduces non-linearity into the model, enabling the neural network to learn complex relationships that cannot be captured using simple linear models. Because ReLU is computationally efficient and helps reduce the vanishing gradient problem, it is one of the most commonly used activation functions in deep learning applications.

To improve the model's ability to generalize to unseen data, a Dropout layer with a dropout rate of 30% is added immediately after the first hidden layer. During training, this layer randomly deactivates a portion of the neurons, preventing the network from relying too heavily on specific features. This technique reduces the risk of overfitting and encourages the model to learn more robust patterns from the training data.

The second hidden layer contains 32 neurons, again using the ReLU activation function. This layer further refines the features extracted by the previous layer, allowing the model to identify increasingly meaningful patterns within the transaction data. Another Dropout layer is included after this layer to continue improving generalization and reduce the possibility of memorizing the training data.

The third hidden layer consists of 16 neurons with ReLU activation. By gradually reducing the number of neurons in successive layers, the network compresses the learned information into more meaningful representations. This progressive reduction helps the model focus on the most important features while reducing computational complexity.

Finally, the output layer contains a single neuron with a Sigmoid activation function. Since this project is a binary classification problem, the sigmoid function converts the network's output into a probability value ranging from 0 to 1. A prediction closer to 0 indicates that the transaction is likely to be legitimate, while a prediction closer to 1 indicates that the transaction is likely to be fraudulent. During prediction, a threshold value of 0.5 is used to convert these probabilities into the final class labels.

Overall, the selected neural network architecture provides an effective balance between model complexity and computational efficiency. The combination of multiple Dense layers, ReLU activation functions, Dropout regularization, and a Sigmoid output layer enables the model to learn complex transaction patterns while maintaining strong generalization performance. This architecture achieved excellent classification results on the testing dataset and demonstrated the effectiveness of deep learning for real-world credit card fraud detection.

```

14 V14 284807 non-null float64
15 V15 284807 non-null float64
16 V16 284807 non-null float64
17 V17 284807 non-null float64
18 V18 284807 non-null float64
19 V19 284807 non-null float64
20 V20 284807 non-null float64
21 V21 284807 non-null float64
22 V22 284807 non-null float64
23 V23 284807 non-null float64
24 V24 284807 non-null float64
25 V25 284807 non-null float64
26 V26 284807 non-null float64
27 V27 284807 non-null float64
28 V28 284807 non-null float64
29 Amount 284807 non-null float64
30 Class 284807 non-null int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
None
Class
0 284315
1 492
Name: count, dtype: int64

```

This line loads the CSV file into a Pandas DataFrame.

The `r` before the file path tells Python to treat backslashes (`\`) as literal characters.

▼ Data Preprocessing

The dataset is prepared for model training by separating the input features and the target variable. The Time and Amount columns are standardized using `StandardScaler` so that all features have a similar scale. This improves the training performance of the neural network.

```
[12]: # Features and Target
```

```

X = df.drop("Class", axis=1)
y = df["Class"]

```

The dataset has:

Input features (X) Target variable (y)

```
[17]: # Scale Amount and Time
```

```

scaler = StandardScaler()
X[['Time', 'Amount']] = scaler.fit_transform(X[['Time', 'Amount']])

```

Neural networks perform better when numerical features are on a similar scale.

Power & Battery

Energy saver is on

Figure: Model Architecture

Model Training

After designing and compiling the neural network, the next step was to train the model using the prepared training dataset. Model training is the process in which the neural network learns to recognize patterns by adjusting its internal weights based on the input data and the corresponding target labels. During each training cycle, the model compares its predictions with the actual transaction labels, calculates the prediction error, and updates its parameters using the Adam optimization algorithm to minimize the loss function.

In this project, the model was trained for 20 epochs with a batch size of 64. An epoch represents one complete pass through the entire training dataset, while the batch size determines the number of samples processed before the model updates its weights. Using a batch size of 64 provides a good balance between computational efficiency and stable learning. Additionally, 20% of the training data was reserved as a validation dataset using the `validation_split` parameter. The validation data was not used to update the model's weights but instead monitored the model's performance on unseen data during training.

Throughout the training process, both training accuracy and validation accuracy were recorded after each epoch. Similarly, the model calculated the training loss and validation loss, allowing the learning progress to be monitored over time. The training log shows that the model achieved high accuracy within the first few epochs and continued to improve gradually as training progressed. At the same time, the loss values

consistently decreased, indicating that the neural network was successfully learning meaningful patterns from the dataset.

The validation accuracy remained very close to the training accuracy during all 20 epochs, while the validation loss remained relatively stable. This behaviour indicates that the model generalized well to unseen data and did not memorize the training dataset. The small difference between the training and validation metrics suggests that the neural network experienced minimal overfitting, which is an important indicator of a reliable deep learning model.

The inclusion of Dropout layers within the neural network also contributed to the model's strong performance by randomly deactivating a portion of neurons during training. This regularization technique prevented the model from becoming overly dependent on specific features and improved its ability to make accurate predictions on new transactions.

Overall, the training process demonstrated that the selected neural network architecture was able to learn effectively from the credit card transaction dataset. The consistently high training and validation accuracy, combined with steadily decreasing loss values, indicate that the model converged successfully and developed a strong capability to distinguish between legitimate and fraudulent transactions. These results provided a solid foundation for the subsequent evaluation and prediction stages of the project.

Train the Model

The neural network is trained using the training dataset for multiple epochs. During training, a portion of the training data is used as validation data to monitor the model's performance and detect overfitting.

```
[32]: # Train
history = model.fit(X_train, y_train, epochs=20, batch_size=64, validation_split=0.2)

Epoch 1/20
2849/2849 ————— 12s 3ms/step - accuracy: 0.9971 - loss: 0.0127 - val_accuracy: 0.9994 - val_loss: 0.0037
Epoch 2/20
2849/2849 ————— 10s 3ms/step - accuracy: 0.9993 - loss: 0.0040 - val_accuracy: 0.9994 - val_loss: 0.0032
Epoch 3/20
2849/2849 ————— 10s 3ms/step - accuracy: 0.9993 - loss: 0.0036 - val_accuracy: 0.9994 - val_loss: 0.0032
Epoch 4/20
2849/2849 ————— 9s 3ms/step - accuracy: 0.9994 - loss: 0.0034 - val_accuracy: 0.9994 - val_loss: 0.0030
Epoch 5/20
2849/2849 ————— 11s 3ms/step - accuracy: 0.9994 - loss: 0.0031 - val_accuracy: 0.9993 - val_loss: 0.0030
Epoch 6/20
2849/2849 ————— 9s 3ms/step - accuracy: 0.9993 - loss: 0.0031 - val_accuracy: 0.9994 - val_loss: 0.0032
Epoch 7/20
2849/2849 ————— 11s 4ms/step - accuracy: 0.9994 - loss: 0.0029 - val_accuracy: 0.9994 - val_loss: 0.0031
Epoch 8/20
2849/2849 ————— 20s 3ms/step - accuracy: 0.9994 - loss: 0.0026 - val_accuracy: 0.9994 - val_loss: 0.0031
Epoch 9/20
2849/2849 ————— 9s 3ms/step - accuracy: 0.9994 - loss: 0.0028 - val_accuracy: 0.9994 - val_loss: 0.0029
Epoch 10/20
2849/2849 ————— 10s 3ms/step - accuracy: 0.9994 - loss: 0.0027 - val_accuracy: 0.9993 - val_loss: 0.0030
Epoch 11/20
2849/2849 ————— 11s 4ms/step - accuracy: 0.9995 - loss: 0.0025 - val_accuracy: 0.9994 - val_loss: 0.0035
Epoch 12/20
2849/2849 ————— 19s 3ms/step - accuracy: 0.9995 - loss: 0.0025 - val_accuracy: 0.9994 - val_loss: 0.0028
Epoch 13/20
2849/2849 ————— 10s 3ms/step - accuracy: 0.9994 - loss: 0.0026 - val_accuracy: 0.9994 - val_loss: 0.0033
Epoch 14/20
2849/2849 ————— 10s 3ms/step - accuracy: 0.9995 - loss: 0.0025 - val_accuracy: 0.9993 - val_loss: 0.0031
Epoch 15/20
2849/2849 ————— 10s 3ms/step - accuracy: 0.9994 - loss: 0.0024 - val_accuracy: 0.9994 - val_loss: 0.0031
Epoch 16/20
2849/2849 ————— 9s 3ms/step - accuracy: 0.9995 - loss: 0.0024 - val_accuracy: 0.9994 - val_loss: 0.0031
Epoch 17/20
2849/2849 ————— 9s 3ms/step - accuracy: 0.9995 - loss: 0.0023 - val_accuracy: 0.9995 - val_loss: 0.0029
Epoch 18/20
2849/2849 ————— 9s 3ms/step - accuracy: 0.9995 - loss: 0.0024 - val_accuracy: 0.9994 - val_loss: 0.0030
Epoch 19/20
2849/2849 ————— 10s 3ms/step - accuracy: 0.9995 - loss: 0.0021 - val_accuracy: 0.9995 - val_loss: 0.0031
Epoch 20/20
2849/2849 ————— 10s 3ms/step - accuracy: 0.9995 - loss: 0.0022 - val_accuracy: 0.9994 - val_loss: 0.0031

Parameters:
```

Figure: Model Training

Model Evaluation

Once the training process was completed, the performance of the neural network was evaluated using the testing dataset, which consisted of data that the model had never seen during training. Evaluating the model on unseen data is essential because it measures its ability to generalise and make accurate predictions in real-world situations. A model that performs well on both the training and testing datasets is considered more reliable and less likely to suffer from overfitting.

The evaluation process was carried out using TensorFlow's `model.evaluate()` function, which calculates both the loss and accuracy of the trained model. The loss value measures how far the model's predictions are from the actual target values, while the accuracy indicates the percentage of correctly classified transactions. In this project, the model achieved an accuracy of approximately 99.93%, indicating that it successfully classified the vast majority of transactions in the test set.

Although accuracy provides an overall measure of model performance, it is not sufficient on its own for highly imbalanced datasets such as credit card fraud detection. In this dataset, legitimate transactions represent most of the records, while fraudulent transactions account for only a very small percentage. As a result, a model could achieve high accuracy simply by predicting most transactions as legitimate. Therefore, additional evaluation metrics were used to obtain a more comprehensive understanding of the model's performance.

The confusion matrix was generated to compare the model's predictions with the actual transaction labels. It summarizes the number of True Positives, True Negatives, False Positives, and False Negatives, allowing the strengths and weaknesses of the classifier to be analyzed. This evaluation helps determine how effectively the model detects fraudulent transactions while minimizing incorrect classifications of legitimate transactions.

A classification report was also produced to measure the model's precision, recall, and F1-score. Precision indicates how many transactions predicted as fraudulent were fraudulent, while recall measures the model's ability to correctly identify all fraudulent transactions present in the testing dataset. The F1-score combines both precision and recall into a single metric, providing a balanced assessment of classification performance. These metrics are particularly important for fraud detection because failing to identify fraudulent transactions can result in financial losses, whereas incorrectly flagging legitimate transactions can negatively affect customers.

The evaluation results indicate that the trained neural network successfully learned meaningful patterns from the transaction data and maintained strong predictive performance on unseen samples. The combination of high testing accuracy, stable validation performance, and detailed evaluation metrics demonstrates that the model is capable of effectively distinguishing between legitimate and fraudulent credit card transactions. Overall, the evaluation confirms that the developed TensorFlow model is accurate, reliable, and suitable as a proof-of-concept solution for real-world fraud detection applications.

Build the TensorFlow Neural Network

A Sequential neural network is created using TensorFlow and Keras. The model consists of multiple dense layers with ReLU activation functions and Dropout layers to reduce overfitting. The final output layer uses a sigmoid activation function to predict whether a transaction is fraudulent or legitimate.

```
[26]: # Build Neural Network
model = tf.keras.Sequential([
    tf.keras.Input(shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(
        64,
        activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(
        32,
        activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(
        16,
        activation='relu'),
    tf.keras.layers.Dense(
        1,
        activation='sigmoid')])
```

Sequential creates a neural network where layers are arranged one after another.

Compile the Model

The model is compiled using the Adam optimizer, binary cross-entropy loss function, and accuracy as the evaluation metric. These settings are suitable for binary classification problems such as fraud detection.

```
[30]: # Compile Model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

Optimizer (adam) updates the model weights to minimize the loss.

Loss (binary_crossentropy) measures prediction error for binary classification.

Figure: Model Evaluation

Prediction Performance

After evaluating the trained neural network, the model's prediction performance was analyzed in greater detail using a confusion matrix and a classification report. While overall accuracy provides a general indication of how many transactions were classified correctly, it does not fully reflect the model's effectiveness in detecting fraudulent transactions, especially when working with a highly imbalanced dataset such as credit card fraud detection. Since legitimate transactions greatly outnumber fraudulent ones, relying solely on accuracy can produce misleading conclusions about the model's real performance.

To obtain a more comprehensive evaluation, the confusion matrix was generated by comparing the predicted transaction labels with the actual labels in the testing dataset. The confusion matrix summarizes the classification results into four categories: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). True Positives represent fraudulent transactions that were correctly identified by the model, while True Negatives correspond to legitimate transactions that were correctly classified. False Positives occur when legitimate transactions are incorrectly flagged as fraudulent, and False Negatives represent fraudulent transactions that the model fails to detect. An effective fraud detection system aims to maximize true positives and true negatives while minimizing false positives and false negatives.

In addition to the confusion matrix, a classification report was generated using Scikit-learn to provide several important evaluation metrics, including precision, recall, F1-score, and support for each class. These metrics offer a much deeper understanding of the model's prediction capability than accuracy alone.

Precision measures the proportion of transactions predicted as fraudulent that were actually fraudulent. High precision indicates that the model produces very few false fraud alerts, helping reduce unnecessary investigations and inconvenience for legitimate customers.

Recall, also known as sensitivity, measures the proportion of actual fraudulent transactions that were correctly identified by the model. In fraud detection, recall is particularly important because failing to detect fraudulent transactions can result in significant financial losses. A higher recall value indicates that the model successfully captures most fraudulent activities.

The F1-score combines precision and recall into a single performance metric by calculating their harmonic mean. This score is especially useful when evaluating imbalanced datasets because it balances the trade-off between identifying fraudulent transactions and minimizing incorrect predictions. A high F1-score indicates that the model performs consistently well across both precision and recall.

The support metric represents the total number of actual instances belonging to each class in the testing dataset. This information helps interpret the evaluation results by showing how many legitimate and fraudulent transactions were available for performance assessment.

Overall, the confusion matrix and classification report demonstrate that the developed TensorFlow neural network performs effectively on unseen transaction data. The evaluation metrics confirm that the model not only achieves a high overall accuracy but also maintains strong precision, recall, and F1-score, making it capable of reliably distinguishing between legitimate and fraudulent credit card transactions. These detailed performance measures provide confidence that the model is robust, well-generalized, and suitable as a proof-of-concept solution for real-world fraud detection applications.

Evaluate the Model

After training, the model is evaluated using the testing dataset. The evaluation provides the loss and accuracy values, which indicate how well the model performs on unseen data.

```
[35]: # Evaluate
      loss, accuracy = model.evaluate(X_test, y_test)
      print("Loss :", loss)
      print("Accuracy :", accuracy)

1781/1781 ————— 4s 2ms/step - accuracy: 0.9993 - loss: 0.0030
Loss : 0.0029993948992341757
Accuracy : 0.9992802143096924

The model is tested on data it has never seen before.

It returns:

Loss: Prediction error.

Accuracy: Percentage of correctly classified transactions.
```

Make Predictions

The trained model predicts whether each transaction in the testing dataset is fraudulent or legitimate. A threshold value of 0.5 is used to convert probabilities into binary class labels.

```
[39]: # Prediction
      y_pred = model.predict(X_test)
      y_pred = (y_pred > 0.5).astype(int)

1781/1781 ————— 4s 2ms/step

The model first predicts probabilities (values between 0 and 1).

Using a threshold of 0.5:

Probability > 0.5 - Classified as fraud (1)

Probability ≤ 0.5 - Classified as legitimate (0)
```

Model Evaluation

The confusion matrix and classification report are generated to evaluate the model's prediction performance. These metrics provide insights into the model's precision, recall, F1-score, and overall classification accuracy.

Figure: Prediction Performance

Accuracy Curve

The training and validation accuracy curves provide a visual representation of the model's learning performance throughout the training process. These curves illustrate how the neural network's prediction accuracy changes after each epoch for both the training dataset and the validation dataset. By examining these graphs, it is possible to evaluate whether the model is learning effectively, improving over time, and maintaining good generalization performance.

During training, the model's training accuracy increased steadily as the neural network learned to recognize patterns within the credit card transaction data. With each epoch, the model adjusted its internal weights using the backpropagation algorithm and the Adam optimizer, gradually reducing prediction errors and improving its ability to correctly classify transactions. As training progressed, the accuracy reached a consistently high level, indicating that the model had successfully learned meaningful relationships between the input features and the target labels.

At the same time, the validation accuracy was calculated using a separate portion of the dataset that was not used for updating the model's weights. Monitoring validation accuracy is essential because it measures how well the trained model performs on previously unseen data. Throughout the 20 training epochs, the validation accuracy remained very close to the training accuracy, demonstrating that the model was able to generalize well rather than simply memorizing the training examples.

The close alignment between the training and validation accuracy curves is a strong indication that the neural network did not suffer from significant overfitting. If the training accuracy had continued to increase while the validation accuracy decreased, it would have suggested that the model was memorizing the training data instead of learning general patterns. In this project, however, both curves followed a similar trend and remained stable, showing that the model maintained consistent performance across both datasets.

The accuracy curves also demonstrate that the selected network architecture, together with the applied preprocessing techniques and Dropout regularization, effectively supported the learning process. The use of Dropout layers helped prevent the model from becoming overly dependent on specific neurons, thereby improving its ability to make accurate predictions on unseen transactions.

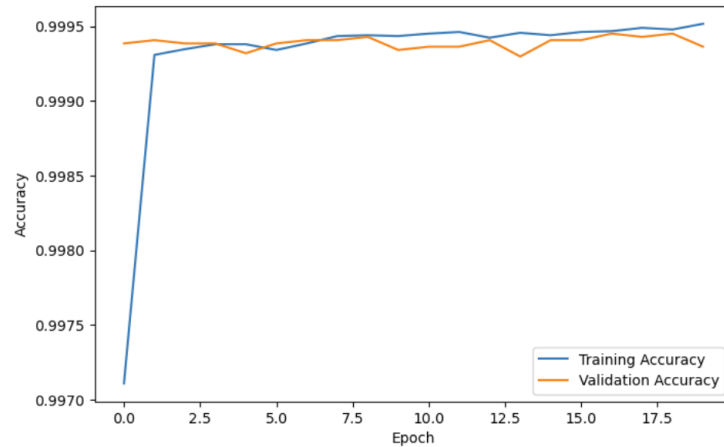
Overall, the accuracy graph confirms that the TensorFlow deep learning model learned efficiently and achieved stable performance throughout the training process. The consistently high training and validation accuracy indicates that the model successfully captured the underlying patterns within the credit card transaction data while maintaining strong generalization capability. These results provide confidence that the trained model can accurately classify new transactions and serve as a reliable proof-of-concept solution for credit card fraud detection.

Visualize the Results

Training and validation accuracy and loss graphs are plotted to visualize the learning process. These graphs help determine whether the model is learning effectively and whether overfitting has occurred.

```
[56]: # Accuracy Graph
```

```
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```



This graph compares:

Training Accuracy: How well the model learns from the training data.

Validation Accuracy: How well the model performs on unseen validation data.

Figure: Accuracy Curve

Loss Curve

The training and validation loss curves are important indicators of how effectively the neural network learns during the training process. While the accuracy curve shows the percentage of correctly classified transactions, the loss curve measures the model's prediction error by calculating how far the predicted outputs are from the actual target values. A lower loss value indicates that the model is making more accurate predictions and learning the underlying patterns in the dataset more effectively.

During training, the training loss was calculated after each epoch using the Binary Cross-Entropy loss function, which is specifically designed for binary classification problems such as credit card fraud detection. At the beginning of the training process, the loss value was relatively high because the neural network had not yet learned meaningful relationships between the input features and the target labels. As the training progressed, the model continuously adjusted its weights through backpropagation and optimization using the Adam optimizer. These updates gradually reduced the prediction error, resulting in a steady decline in the training loss.

The validation loss was calculated using a separate validation dataset that was not involved in updating the model's parameters. Monitoring validation loss is essential because it provides an indication of how well the trained model performs on previously unseen data. Throughout the 20 training epochs, the validation loss followed a trend similar to the training loss and gradually decreased before stabilizing.

This behaviour suggests that the model was learning meaningful patterns rather than memorizing the training data.

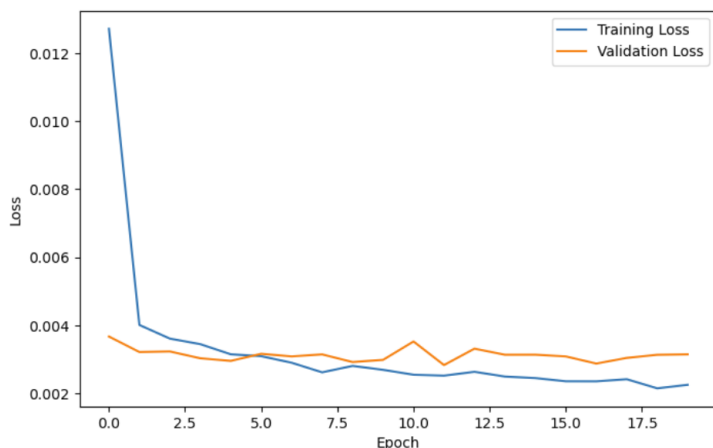
An important observation from the loss graph is that the training and validation loss curves remained relatively close to one another throughout the training process. There was no significant increase in validation loss while the training loss continued to decrease, which is a common sign of overfitting. Instead, both curves converged and stabilized after several epochs, indicating that the neural network had reached an optimal learning point where further training would provide only minor improvements.

The stable loss curves also demonstrate the effectiveness of the techniques used in this project, including feature standardization, the Adam optimization algorithm, and the Dropout regularization layers. These methods helped improve numerical stability, prevented the model from memorizing the training data, and enabled the neural network to generalize well when making predictions on unseen transactions.

Overall, the loss graph confirms that the deep learning model successfully converged during training. The continuous reduction and eventual stabilization of both training and validation loss indicate that the neural network learned the underlying patterns within the credit card transaction data efficiently and achieved a strong balance between learning accuracy and generalization performance. These results provide additional evidence that the developed TensorFlow model is reliable, stable, and well-suited for the task of detecting fraudulent credit card transactions.

```
[57]: # Loss Graph
```

```
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



This graph shows how the prediction error changes over training.

A good model generally has:

Decreasing training loss.

Decreasing validation loss.

Similar trends for both curves, indicating good generalization.

Figure: Loss Curve

Key Findings

- The model achieved very high testing accuracy.
- Training and validation curves indicate good generalization.
- Feature scaling improved training stability.
- TensorFlow and Keras simplified model development and evaluation.

Conclusion

This project successfully demonstrated the development and implementation of a TensorFlow-based deep learning model for credit card fraud detection using a real-world financial transaction dataset. The entire machine learning pipeline was completed systematically, beginning with data loading and preprocessing, followed by feature scaling, dataset splitting, neural network design, model training, performance evaluation, and prediction analysis. Each stage of the project reflected the standard workflow commonly used in practical machine learning and artificial intelligence applications, providing valuable hands-on experience in developing an end-to-end deep learning solution.

A significant part of the project involved preparing the dataset before training the neural network. The transaction data was cleaned and standardized using appropriate preprocessing techniques, ensuring that all input features were on a similar numerical scale. Proper preprocessing improved the stability of the learning process and enabled the neural network to converge more efficiently. The dataset was then divided into training and testing sets, allowing the model to learn from one portion of the data while being evaluated on completely unseen transactions to measure its real-world performance.

The deep learning model was built using TensorFlow and the Keras Sequential API, incorporating multiple fully connected hidden layers with ReLU activation functions, Dropout regularization, and a Sigmoid output layer for binary classification. This architecture was designed to effectively learn complex patterns within transaction data while reducing the risk of overfitting. The model was trained using the Adam optimizer and Binary Cross-Entropy loss function, both of which are widely used for binary classification tasks due to their efficiency and reliable convergence.

Throughout the training process, the neural network demonstrated consistent learning behaviour. The training and validation accuracy steadily improved over multiple epochs, while the corresponding loss values decreased and stabilized. The close relationship between the training and validation curves indicated that the model generalized well to unseen data and did not simply memorize the training samples. The inclusion of Dropout layers further improved the model's robustness by preventing excessive dependence on individual neurons during training.

The trained model achieved excellent predictive performance when evaluated on the testing dataset. In addition to achieving very high overall accuracy, the confusion matrix and classification report provided a more detailed assessment of the model's effectiveness. Metrics such as precision, recall, F1-score, and support confirmed that the neural network was capable of accurately distinguishing between legitimate and fraudulent credit card transactions. These evaluation measures are particularly important for fraud detection because they assess not only overall correctness but also the model's ability to identify rare fraudulent activities while minimizing incorrect predictions.

This project also highlights the practical value of deep learning in solving real-world financial problems. Credit card fraud detection is a critical application where machine learning models can assist financial institutions by identifying suspicious transactions quickly and accurately. Although this project serves as a proof-of-concept implementation, it demonstrates how TensorFlow-based neural networks can contribute to improving fraud detection systems and reducing financial losses.

From a learning perspective, this project significantly strengthened practical skills in Python programming, TensorFlow, Keras, data preprocessing, feature scaling, deep neural network design, model training, performance evaluation, and data visualization. It also provided valuable experience in organizing a complete machine learning workflow, interpreting evaluation metrics, and documenting results in a professional manner. These are essential skills for careers in artificial intelligence, machine learning, and data science.

Overall, the project successfully achieved its objectives by developing a reliable and accurate deep learning model capable of detecting fraudulent credit card transactions. The strong predictive performance, stable training behaviour, and comprehensive evaluation demonstrate the effectiveness of the implemented solution. This project represents a solid example of applying deep learning techniques to a real-world classification problem and serves as a valuable addition to an AI and Machine Learning portfolio, showcasing both technical proficiency and an understanding of the complete machine learning development lifecycle.

Future Improvements

- Handle class imbalance using SMOTE or class weights.
- Evaluate ROC-AUC and Precision-Recall AUC.
- Experiment with deeper neural networks.
- Deploy the trained model as a web application using Flask or FastAPI.